

Exercices chapitre 2-Programmation fonctionnelle

Un des objectifs de cette feuille d'exercice est de s'habituer à la programmation fonctionnelle. Donc, sauf mention explicite du contraire, pas de boucle, encore moins de références...

Exercice 2.1

Étude de deux premières fonctions

1) On considère la fonction:

```
let rec affiche n =
  if n = 0 then
    begin
      print_string "j ai fini";
      print_newline();
    end
  else
    begin
      print_int n;
      print_string " ";
      affiche (n-1);
      print_int (n*n);
      print_string " ";
    end;
end;;
```

- Montrer qu'un appel de `affiche n` s'arrête lorsque n est un entier naturel.
- Expliquer ce que va faire un tel appel.
- Justifier-le avec l'arbre des appels récursifs.
- Justifier-le par récurrence.
- Vérifier en testant cette fonction.
- Calculer sa complexité en nombre d'impressions à l'écran.
- Calculer sa complexité en nombre d'appels récursifs.

2) Écrire des fonctions `triangle : int -> unit` et `triangle_inv : int -> unit` qui affichent à l'écran des triangles d'étoiles *. Par exemple, `triangle 5` doit afficher:

```
*
**
***
****
*****
```

et `triangle_inv 7` doit afficher

```
*****
*****
*****
****
***
**
*
```

Quelle est la complexité de ces fonctions?

Exercice 2.2

Que font les fonctions suivantes? preuve?

1)

```
let rec f n = (n = 0) || g (n - 1)
and g n = (n <> 0) && f (n - 1) ;;
```

2)

```
let rec f n = if n = 0 then 1 else 3 + g (n-1)
and g n = if n = 0 then 0 else 1 + f (n-1);;
```

Exercice 2.3

On définit les fonctions suivantes:

```
let succ x = x + 1;;
let pred x = x - 1;;
```

En déduire à l'aide d'une approche purement fonctionnelle:

- 1) La fonction `add` de type `int -> int -> int` qui ajoute deux entiers relatifs.
- 2) En déduire une fonction `mult` de type `int -> int -> int` qui multiplie deux entiers relatifs. (On s'autorisera d'obtenir l'opposé de x simplement par $-x$).

Exercice 2.4

Fonction récursive simple de la variable entière

- 1) Programmer une fonction `calcul_u phi n` a récursive de calcul de u_n pour $n \in \mathbb{N}$ lorsque u est la suite définie par: $u_0 = a$ et $\forall n \in \mathbb{N}, u_{n+1} = \varphi(u_n)$. Quel est le type de cette fonction?
- 2) Modifier votre fonction pour qu'elle affiche tous les termes de la suite u de u_0 à u_n .
- 3) Tester votre fonction avec les deux suites suivantes:
 - a) $u_0 = 2$ et $\forall n \in \mathbb{N}, u_{n+1} = 2 + u_n$.
 - b) $u_0 = ""$ (chaîne vide) et $u_{n+1} = u_n \wedge "ab"$.
- 4) Préciser sa complexité en nombre d'appels de φ .
- 5) Comparer la complexité avec une version itérative.

Exercice 2.5

Suites récurrentes du premier ordre

Dans chacun des cas suivants, déterminer une fonction récursive de calcul de u_n . Évaluer la complexité en nombre d'opérations. Comparer avec une version itérative.

- 1) $u_0 = a \in \mathbb{R}$ et $\forall n \in \mathbb{N}^*, u_{n+1} = \sqrt{2u_n + n}$.
- 2) u_0 est la chaîne vide et $\forall n \in \mathbb{N}^*, u_n$ est la concaténation dans cet ordre de u_{n-1} , de "n", de u_{n-1} .

Exercice 2.6

Fonction récursive multiple de la variable entière

- 1) Programmer une fonction récursive de calcul de v_n où v est la suite définie par: $v_0 = a, v_1 = b$ et $v_{n+2} = \varphi(v_n, v_{n+1})$.
- 2) Modifier votre fonction pour qu'elle affiche tous les termes de la suite v de v_0 à v_n .
- 3) Tester votre fonction dans les deux cas suivant:
 - a) $v_0 = 1, v_1 = 1$ et $\forall n \in \mathbb{N}, v_{n+2} = v_n + v_{n+1}$.
 - b) v_0 est la chaîne vide, $v_1 = "a"$ et $v_{n+2} = v_n \wedge "ab" \wedge v_{n+1}$.
- 4) Préciser la complexité de cette fonction en nombre d'appels de φ .

Exercice 2.7

Syracuse

Il est conjecturé que la suite de premier terme $u_0 = a \in \mathbb{N}^*$ et telle que

$$\forall n \in \mathbb{N}, \begin{cases} u_{n+1} = \frac{u_n}{2} & \text{si } u_n \text{ est pair} \\ u_{n+1} = 3u_n + 1 & \text{sinon} \end{cases}.$$

finit par prendre la valeur 1, on appelle alors orbite de a les suites des valeurs prises par la suite $(u_n)_{n \in \mathbb{N}}$ jusqu'à la valeur 1 lorsque $u_0 = a$.

Coder une fonction de type `int -> unit` telle que `syracuse a` affiche l'orbite de la suite de Syracuse de premier terme $u_0 = a$.

Coder une fonctions `int -> int list` qui renvoie l'orbite de la suite de Syracuse d'un entier naturel passé en argument.

Exercice 2.8

Écrire de manière purement fonctionnelle:

- 1) Une fonction de calcul d'une approximation de $\int_a^b \phi(x)dx$ par méthode des rectangles:
(Fonction d'en-tête `integrale phi a b dx` de type `(float -> float) -> float -> float -> float -> float`).
- 2) Une fonction de calcul de $\sum_{k=a}^b f(k)$ lorsque f est une fonction de $\mathbb{N}$ vers $\mathbb{N}$.
(Fonction d'en-tête `somme f a b` de type `(int -> int) -> int -> int -> int`).
- 3) Une fonction qui vérifie si un entier x est un des termes de la suite de Fibonacci.
(Fonction d'en-tête `dansFibo x` de type `dansFibo : int -> bool`).
- 4) Une fonction de calcul de $\lfloor \sqrt{n} \rfloor$ lorsque $n \in \mathbb{N}$. On écrira une fonction de complexité logarithmique.
(Fonction d'en-tête `racineEntiere n` de type `int -> int`).

Exercice 2.9

Suites récurrentes du second ordre ou plus

- 1) Programmer une fonction récursive de calcul de v_n lorsque $\forall n \in \mathbb{N}, v_{n+3} = \sqrt{v_n v_{n+1} v_{n+2}}$ et $v_0, v_1, v_2 = a, b, c$.
- 2) Plus généralement, étudier la complexité d'une programmation récursive de la fonction `u f n` lorsque u est une suite récurrente d'ordre p , telle que $\forall n \in \mathbb{N}, u_{n+p} = f(u_n, \dots, u_{n+p-1})$.
- 3) Programmer une fonction récursive de calcul de $(v_n)_{n \in \mathbb{N}}$ lorsque $\forall n \in \mathbb{N}, v_{n+1} = \sum_{k=0}^n v_k v_{n-k}$ et $v_0 = 1$.

Exercice 2.10

Suites récurrentes, approche fonctionnelle

- 1) a) Écrire une fonction `iterer` de type `('a -> 'a) -> int -> 'a -> 'a` telle que `iterer f n` renvoie la fonction codant $f \circ \dots \circ f$ (n fois).
b) Vérifiez que votre fonction ne provoque pas de dépassement de pile lorsque l'on itère un million de fois la fonction $x \mapsto x + 1$.
- 2) En déduire une fonction `fibonacci` qui renvoie le n -ème terme de la suite de Fibonacci.

Exercice 2.11

Exponentiation rapide

On a pour tout $n \in \mathbb{N}$ et tout $x \in \mathbb{R}$:

$$x^n = x^{(n \text{ div } 2) \times 2 + (n \text{ mod } 2)}.$$

$$x^n = x^{(n \text{ div } 3) \times 3 + (n \text{ mod } 3)}.$$

- 1) En déduire deux fonctions de calcul de x^n .
- 2) Prouver la validité de ces fonctions.
- 3) Comparer la complexité de ces deux fonctions.

Exercice 2.12

Récursivité sur la taille d'un problème: les tableaux

Soit t un tableau d'entiers de n cases.

- 1) Écrire une fonction `echange t i j` qui échange les contenus des cases d'indices i et j du tableau t . (les indices i et j sont supposés être des indices valides de t).
- 2) On veut calculer la somme des éléments de t .
a) Écrire une fonction récursive qui calcule la somme des éléments de t des cases 0 à j .
b) Préciser sa complexité en nombre d'additions.
c) En déduire une fonction qui calcule la somme des éléments de t . Préciser sa complexité.
- 3) On veut calculer le maximum des éléments de t .
a) En vous inspirant de ce qui précède, écrire cette fonction. Elle pourra faire appel à une fonction auxiliaire qui est récursive.
b) Préciser sa complexité.
- 4) Soit x un entier.
a) En vous inspirant de ce qui précède, écrire une fonction qui retourne le booléen `true` si et seulement si x est dans t . Cette fonction devra arrêter la recherche dès que x est trouvé. Elle pourra faire appel à une fonction auxiliaire qui est récursive.

b) Préciser sa complexité en nombre de comparaisons avec x .

c) Modifier la fonction pour qu'elle renvoie un indice de position de x dans le tableau et (-1) si x n'est pas dans le tableau.

d) On suppose que maintenant que le tableau t est trié par ordre croissant. Écrire une nouvelle fonction en $\theta(\ln(n))$ qui renvoie la valeur du booléen $x \in t$.

5) Soient t et t' deux tableaux de même taille. On dit que t' est une anagramme de t si t' se déduit de t par permutation des éléments de t . Écrire une fonction (`anagramme t tp`) qui renvoie le booléen `true` si et seulement si t' est une anagramme de t .

Exercice 2.13
Récursivité sur la taille d'un problème: les chaînes de caractères

Soit w une chaîne de caractères de longueur n .

1) Soit w' une autre chaîne. On veut savoir si w' est préfixe de w . On note n' la longueur de w' .

a) On suppose que $n' = 0$. Que doit-on renvoyer ? `true` ou `false`.

b) On suppose que $n' \neq 0$ et $n = 0$. Que doit-on renvoyer ?

c) On suppose que n et n' sont non nuls. Le mot w a donc une première lettre x et w' une première lettre x' . Que renvoie-t-on si $x \neq x'$? Que fait-on si $x = x'$?

d) En déduire une fonction CAML `est_prefixe wp w`.

e) Que se passe-t-il si $n' > n$? Renvoie-t-on le bon résultat ? Pourquoi ?

f) Évaluer la complexité de votre fonction en nombre d'appels récursifs.

g) Comparer avec une version itérative.

2) Écrire de la même façon une fonction `est_suffixe wp w`.

3) Écrire une fonction `est_facteur wp w`. Évaluer sa complexité. Rappel: w' est dit facteur de w si il existe deux mots u et v tels que : $w = u \wedge w' \wedge v$. On pourra utiliser la fonction `est_prefixe`.

4) Écrire une fonction `miroir w` qui renvoie le mot w dans l'ordre inverse de ses lettres. Par exemple: `miroir "abc"` renvoie `"cba"`. Comparer avec une version itérative.

Exercice 2.14
Combinaisons

On peut définir les combinaisons $\binom{n}{p}$ pour tout $n \in \mathbb{N}$ et $p \in \mathbb{N}$ par les formules:

$$\forall n \in \mathbb{N}^*, \forall p \in \mathbb{N}^*, \binom{n}{p} = \binom{n-1}{p} + \binom{n-1}{p-1}.$$

$$\forall p \in \mathbb{N}^*, \binom{0}{p} = 0, \forall n \in \mathbb{N}, \binom{n}{0} = 1.$$

1) Montrer que l'on définit bien ainsi une fonction sur $\mathbb{N}^2$ et que $\binom{n}{p} = \begin{cases} \frac{n!}{p!(n-p)!} & \text{si } n \geq p \\ 0 & \text{sinon} \end{cases}$.

2) En déduire une programmation récursive de la fonction d'en-tête `comb (n:int) (p:int):int` qui renvoie $\binom{n}{p}$. Évaluer sa complexité en nombre d'appels récursifs puis en nombre d'additions.

3) Mêmes questions en s'appuyant sur la relation valable pour tous $n \in \mathbb{N}$ et $p \in \mathbb{N}$ tels que $1 \leq p \leq n$: $\binom{n}{p} = \frac{n}{p} \binom{n-1}{p-1}$.

Exercice 2.15
Algorithmes de tri-quadratiques

Soit t un tableau d'entiers de n cases.

1) Le tri par sélection

a) Écrire une fonction récursive `pos_min t i` qui renvoie l'indice du minimum de $t.(i..n-1)$.

b) En déduire une fonction récursive `trier_aux t i` qui trie le sous-tableau $t.(i..n-1)$ par effets de bords.

c) En déduire une fonction `trier t` qui trie le tableau t par effets de bords.

d) Évaluer sa complexité.

e) Ce tri est-il stable?

2) Le tri par insertion.

- a) Écrire une fonction récursive `insérer t i` qui insère par effets de bords l'élément $t.(i)$ dans le sous-tableau $t.(0..i-1)$. Le sous-tableau $t.(0..i-1)$ est supposé trié avant l'appel à `insérer t i` et le sous-tableau $t.(0..i)$ doit être trié après l'appel à `insérer t i`.
- b) En déduire une fonction récursive `trier2_aux t i` qui trie le sous-tableau $t.(0..i)$ par effets de bords.
- c) En déduire une fonction `trier2 t` qui trie le tableau t par effets de bords.
- d) Évaluer sa complexité?
- e) Ce tri est-il stable?

Exercice 2.16**Tris quasilinéaires**

- 1) Programmer le tri par fusion d'un tableau.
- 2) Programmer le quicksort. (On utilisera une fonction de partitionnement itérative).

Exercice 2.17**La fonction de Hofstadter**

La fonction de Hofstadter est définie par: $G(n) = 0$ si $n = 0$ et $G(n) = n - G(G(n-1))$ si $n \in \mathbb{N}^*$.

- 1) Montrer que G est définie sur $\mathbb{N}$.
- 2) Écrire une fonction calculant $G(n)$.

Exercice 2.18**Fonctions mal définies**

- 1) Montrer que la fonction récursive suivante n'est pas bien définie sur $\mathbb{N}$

```
let rec sans-fin = fun n ->
  if n = 1 then 1
  else
    if n mod 2 <> 0 then sans-fin(n-1)*n;
    else sans-fin(n-2)*n ;;
```

- 2) La fonction suivante est-elle bien définie sur $\mathbb{N}^2$?

```
let rec N = fun (n,p) -> if n = 0 then 1 else N(n-1,N(n-p,p));;
```

Exercice 2.19**Récursivité croisée**

- 1) Que font les procédures `inconnue1` et `inconnue2` suivantes:

```
let rec inconnue1 = fun n ->
  if n <= 0
  then print_newline()
  else begin
    print_int(n);
    inconnue2(n-1);
    print_int(n) end;
and
  inconnue2 = fun n ->
    if n <= 0 then print_int(n)
    else begin
      print_int(n);
      inconnue1(n-1);
      end;;
```

- 2) Soient a et b deux réels strictement positifs. On considère les suites u et v définies par: $u_0 = a$, $v_0 = b$ et $\forall n \in \mathbb{N}, u_{n+1} = \sqrt{u_n v_n}$, $v_{n+1} = \frac{u_n + v_n}{2}$.

- a) Écrire deux fonctions récursives croisées (sur le modèle de l'exemple précédent) `u a b n` et `v a b n` qui calculent u_n et v_n .
- b) Déterminer leur complexité en nombre de calculs de racines et de divisions.
- c) Donner une nouvelle fonction récursive `uv a b n` qui calcule et renvoie le couple (u_n, v_n) . Déterminer sa complexité en nombre de calculs de racines et de divisions.

Exercice 2.20**Calcul du pgcd de deux entiers**

On considère:

La fonction ϕ de $E = \mathbb{N} \times \mathbb{N}^*$ dans $F = \mathbb{N}^2$ définie par: $\phi(n, p) = (q, r)$ où q (resp. r) est le quotient (resp. le reste) dans la division euclidienne de n par p .

La fonction g de E dans N définie par : $g(a, b) = \text{pgcd}(a, b)$.

1) Soit la fonction récursive f définie par: $f(n, p) = (0, n)$ si $n < p$ et $[f(n - p, p) + (1, 0)]$ sinon. Établir que f est bien définie sur E avec $\forall (n, p) \in E, f(n, p) = \phi(n, p)$.

2) On rappelle que si $a = bq + r$ alors $a \wedge b = b \wedge r$. En déduire une fonction récursive $g(a, b)$ qui calcule $\text{pgcd}(a, b)$. On justifiera sa définition et sa validité.

3) En déduire deux fonctions CAML pour calculer **f** et **g**.

Exercice 2.21

La fonction d'Ackermann

La fonction d'Ackerman est une fonction A de trois variables n, x, y définie par:

- i) $A(0, x, y) = x + 1$
- ii) $A(n, x, 0) = x$ si $n = 1$, 0 si $n = 2$, 1 si $n = 3$ et 2 si $n > 4$.
- iii) $A(n, x, y) = A(n - 1, A(n, x, y - 1), x)$ si $n > 0$ et $y > 0$.

1) Montrer que cette fonction est définie sur $\mathbb{N}^3$ et que l'on a:

$$i) A(1, x, y) = x + y \quad ii) A(2, x, y) = x.y \quad iii) A(3, x, y) = xy.$$

2) Écrire une fonction calculant $A(n, x, y)$.

Exercice 2.22

Les tours de Hanoï

Le jeu des tours de Hanoï est constitué de 3 tiges A, B, C . Sur la tige A sont empilés n disques de diamètres croissants (du bas vers le haut). Le but est de déplacer la pile de disques de la tige A vers la tige B en déplaçant les disques l'un après l'autre mais sans que jamais un disque ne repose sur un disque de diamètre inférieur. C'est-à-dire de sorte que les disques forment toujours une pyramide sur chaque tige. On a l'opération: **DEPLACER(X, Y)** qui déplace le disque au sommet de la tige X vers le sommet de la tige Y .

1) Écrire une procédure **Hanoi A B C n** qui résout le problème. Cette fonction fera appel à la fonction **DEPLACER**. On supposera que **DEPLACER(X, Y)** affiche à l'écran un message du type:

"je déplace un disque de la tour X vers la tour Y".

2) Représenter l'arbre des appels récursifs de **Hanoi A B C 4**.

3) Évaluer la complexité (en nombre d'appels de la fonction **DEPLACER**) de **Hanoi A B C n** en fonction du nombre n .

Exercice 2.23

Algorithmes de multiplication entière

On considère un entier $n \geq 0$ et un réel x . On veut calculer $P(n) = n.x$.

1) Écrire une fonction **multi1 x n** utilisant la définition récursive: $\forall n \in \mathbb{N}^*, P(n) = P(n - 1) + x$. Calculer sa complexité. (**multi1: float -> int -> float**).

2) Exprimer $P(n)$ en fonction de $P(n/2)$ si n est pair et $P((n - 1)/2)$ si n est impair.

3) En déduire une fonction récursive **multi2 x n**. (**multi2: float -> int -> float**). On note $C(n)$ sa complexité, m la complexité d'une multiplication et a celle d'une addition. Exprimer $C(n)$ en fonction de $C(q)$ et de r où q est le quotient dans la division euclidienne de n par 2, et r le reste.

4) On note $n = \sum_{i=0}^{p-1} c_i 2^i$ en base 2. Exprimer $C(n)$ en fonction de p et des c_i .

5) En déduire que la complexité de **multi2** est en $\theta(\ln_2(n))$.

Exercice 2.24

Exponentiation rapide généralisation

1) On se donne une loi interne associative $*$ sur un ensemble E . On suppose que $*$ admet un élément neutre. Écrire une fonction d'en-tête **expo_rapide operateur x n neutre** et de type $('a \rightarrow 'a \rightarrow 'a) \rightarrow 'a \rightarrow \text{int} \rightarrow 'a \rightarrow 'a$ qui calcule x^n lorsque x est un objet de E et $n \in \mathbb{N}$. On suppose que **operateur** code la loi $*$ et **neutre** représente le neutre. On fera en sorte que le nombre d'appels à **operateur** soit en $\Theta(\ln(n))$.

2) En déduire une fonction de calcul de x^n lorsque $n \in \mathbb{N}$ et $x \in \mathbb{R}$.

- 3) D  duire de 1) une fonction de calcul de $n.x$ lorsque $n \in \mathbb{N}$ et $x \in \mathbb{R}$.
- 4) D  duire de 1) une fonction de calcul de x^n lorsque $x \in \mathbb{C}$ et $n \in \mathbb{N}$. (On aura pr  alablement d  fini le `type complexe = float * float`).
- 5) a)   crire la fonction d'en-t  te `prod_matriciel a b` qui calcule le produit de matrices d'entiers carr  es A et B repr  sent  es respectivement par `a` et `b`.
(La fonction `prod_matriciel` sera de type `int array array -> int array array -> int array array`).
- b)   crire une fonction `identite` telle que `identite n` renvoie I_n .
- c) D  duire de 1) une fonction de calcul de A^n lorsque $A \in M_n(\mathbb{Z})$.
- d) **Application: Fibonacci**
- i) On note $(f_n)_{n \in \mathbb{N}}$ la suite de Fibonacci.
- ii) D  terminer une matrice $F \in M_2(\mathbb{Z})$ telle que $\forall n \in \mathbb{N}^*, \begin{pmatrix} f_n \\ f_{n+1} \end{pmatrix} = A \begin{pmatrix} f_{n-1} \\ f_n \end{pmatrix}$.
- iii) Exprimer pour tout $n \in \mathbb{N}$, $\begin{pmatrix} f_n \\ f_{n+1} \end{pmatrix}$    l'aide de A^n .
- iv) Utiliser alors `expo_rapide` pour calculer f_n avec un co  t logarithmique.
- 6) Utiliser `expo_rapide` pour calculer σ^n lorsque σ est une permutation de $\llbracket 1, n \rrbracket$.   valuer le co  t de cette m  thode. Comparer    d'autres m  thodes.

Exercice 2.25	��l��ment majoritaire
----------------------	------------------------------

On appelle   l  ment majoritaire d'un tableau de longueur n , en   l  ment qui appara  t dans un nombre de cases strictement sup  rieur    $\frac{n}{2}$.   crire une fonction qui d  termine s'il existe un   l  ment majoritaire dans un tableau donn  . Prouver la validit   de cette fonction. Estimer sa complexit  .

Exercice 2.26	Nombre d'inversions
----------------------	----------------------------

Dans un tableau d'entiers, on dit que le couple $(i, j) \in \llbracket 0, n-1 \rrbracket$ tel que $i < j$ est une inversion si $t.(i) > t.(j)$.   crire une fonction de calcul du nombre d'inversions d'un tableau `t`.

Exercice 2.27	Probl��me de Reines
----------------------	----------------------------

Le probl  me des 8 reines consiste    placer 8 reines sur un   chiquier de fa  on    ce qu'aucune des reines ne menace une des autres reines. (Une reine A menace une reine B si AB sont sur la m  me diagonale, la m  me verticale, ou la m  me horizontale). R  soudre le probl  me des 8 reines et plus g  n  ralement des n reines sur un   chiquier $n \times n$.

Exercice 2.28	La courbe de Koch
----------------------	--------------------------

La courbe de Koch d'ordre 0 est un segment $[OA]$ avec $O = (0, 0)$ et $A = (a, 0)$. On fabrique la courbe de Koch d'ordre 1 en effectuant les op  rations suivantes:

1. On divise $[OA]$ en trois segments d'  gale longueur: $[OI]$, $[IJ]$ et $[JA]$.
2. On remplace le segment $[IJ]$ par un triangle   quilat  ral direct IJK .
3. On efface le segment $[IJ]$.

La courbe de Koch d'ordre n est alors obtenue en r  p  tant ces op  rations sur chacun des segments de la courbe d'ordre $n-1$.

- 1)   crire une fonction qui r  alise le dessin de la courbe de Koch d'ordre n . Quelle est sa complexit   ?
- 2) M  me question mais la courbe de Koch d'ordre 0 est un triangle   quilat  ral ABC .

Formulaire biblioth  que graphique:

`#load "graphics.cma";; open Graphics;; open_graph"";;` ouverture de la biblioth  que et de la fen  tre graphique.
`size_x();; size_y();;` renvoie le nombre de colonnes, de lignes de pixels, de la fen  tre.
`moveto x1 y1;;` d  place la pointe du crayon vers le point de coordonn  es x_1 et y_1 .
`lineto x2 y2;;` trace un segment de droite de la position courante de la pointe du crayon vers le point de coordonn  es (x_2, y_2) .

- 3) Tracer un flocon de Koch d'ordre n .

Exercice 2.29**Quasi pavage d'un échiquier**

On considère un carré de deux cases consécutives auquel on enlève une case. On obtient alors quatre motifs identiques à rotation près que l'on note M_1 , M_2 , M_3 et M_4 .

On considère un échiquier de taille $2^k \times 2^k$ cases auquel on enlève une case. Montrer qu'il est possible de recouvrir les $2^{2k} - 1$ cases restantes par les motifs M_1 , M_2 , M_3 et M_4 .

Écrire un algorithme effectuant ce pavage.

Exercice 2.30**Sudoku**

Écrire un programme qui complète une grille de Sudoku codée par une matrice d'entiers.

Exercice 2.31**Problème du cavalier**

Écrire un programme qui renvoie un parcours de toutes les cases d'un échiquier par un cavalier, chaque case ne devant être visitée qu'au plus une fois.